Systems Security Assignment I

Mihai Bivol, Daniel Avram, Bence Soóki-Tóth

February 2026

Abstract

In this report we present the solution to Task 1 and Task 4 (Mini TLS-based VPN tunneling). We provide a detailed overview of our attacks, preliminaries and implementation.

Contents

1 Introduction **1**

2 Task 1: ICMP Tunneling **1**
2.1 How ICMP is constructed 1
2.2 How we implemented ICMP 2
2.3 Encryption 4
2.4 How we simulated the proposed exfiltration scenario 4

3 Task 4: Mini TLS-based VPN tunneling **4**
3.1 Prerequisites 4
3.2 How we implemented the VPN Tunneling 6
3.3 Certificate structure 6
3.4 Proof of Work 6

1 Introduction

The goal of this report is to demonstrate how implementations of cryptographic and network protocols can be manipulated to bypass security guarantees or provide secure communication channels. The first part will focus on implementing an one-way encrypted covert channel using the ICMP (Internet Control Message Protocol) protocol to send sensitive data from inside the network without being detected. The second part tackles confidentialy of a VPN tunneling by creating certifications and a TLS tunnel between VPN client and VPN server.

2 Task 1: ICMP Tunneling

2.1 How ICMP is constructed

ICMP, or Internet Control Message Protocol, is a protocol built over IP. The messages sent using ICMP are usually for diagnostic or control purposes, or as responses to errors in IP operations. As such, ICMP packages are directed to an IP address, without the need of a port number. Because

of that, ICMP is also considered a network layer protocol, in contrast to UDP and TCP protocols, (even if all three are built over IP).

The ICMP is encapsulated in an IPv4 packet. There is also a version for IPv6 called ICMPv6, but that is out of the scope. ICMP consists of a header and a payload. It is identified by protocol number 1, and its 8-byte header consists of the following fields:

Type (8 bits)

Code (8 bits)

Checksum (16 bits) calculated from the ICMP header plus the payload, with 0 substituting this field.

Extended Header (32 bits) additional field with varying content based on the chosen type and code.

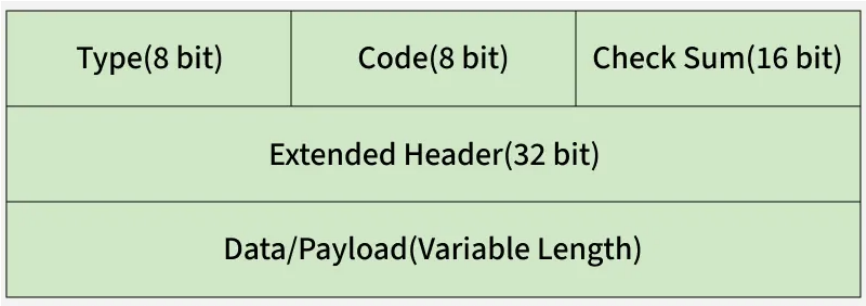

Figure 1: ICMP packet format

Since ICMP packets are used for diagnostics and error handling, firewalls usually don't block them. This leaves them vulnerable to ICMP tunneling, when a service from inside is able to send messages to someone from outside through ICMP traffic, thus bypassing the firewall.

2.2 How we implemented ICMP

Since we are working with IP protocols on a low-level, we chose to implement the solution in python using `socket` library. To set up an ICMP connection, we create a socket object in raw mode (to bypass the checks that are made on IP protocols), and with protocol type set to ICMP:

```
import socket
sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
```

Then, using `struct` package, we create a simple ICMP package with `type` set to 47, `code` set to 0, and an empty extended header:

```
import struct
ICMP_TYPE = 47
def create_packet(payload: bytes) -> bytes:
  # compose header with checksum = 0
  header = struct.pack("!BBHHH", ICMP_TYPE, 0, 0, 0, 0)
```

```
  # calculate checksum
  checksum = calculate_checksum(header + payload)
  # add the checksum to the header
  header = struct.pack("!BBHHH", ICMP_TYPE, 0, checksum, 0, 0)
  # return the packet
  return header + payload
```

The checksum function calculates the **IPv4 header checksum** of the header + payload:

```
def calculate_checksum( data):
  checksum = 0
  # Handle odd-length data by padding with byte 0
  if len(data) % 2 != 0:
      data += b"\x00"
  # Calculate checksum
  for i in range(0, len(data), 2):
      checksum += (data[i] << 8) + data[i+1]
  checksum = (checksum >> 16) + (checksum & 0xffff)
  checksum += checksum >> 16
  return (~checksum) & 0xffff
```

Finally, the packet is sent to the provided client address. The chosen port number is 0, since ICMP does not need one:

```
sock.sendto(packet, (CLIENT, 0))
```

On the receiving side, we open a socket with the same props and bind it to its IP address.

```
sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
sock.bind((HOST, PORT))
```

Then, in an infinite while loop, we receive the message with `recvfrom`, get the protocol type and payload offset from the IP header, and based on the offset, retrieve the data:

```
while True:
  data, _ = sock.recvfrom(1024)
  ip_header = data[:20]
  protocol_number, offset = get_protocol_type_and_offset(ip_header)
  if protocol_number == 1: # for ICMP
      offset += 4 # move offset by ICMP header length
      ciphertext = data[offset: ]

```

The `get_protocol_type_and_offset` function just unpacks the IP header, and from there gets the `protocol number` and the `ihl` that points to the start of the IP packet payload.

2.3 Encryption

In the described scenario, both the sender and the listener share a symmetric key that they use to encrypt the shared messages. We used the `cryptography` package and its `Fernet` encryption that uses `AES256-GCM`:

```
from cryptography.fernet import Fernet
cipher = Fernet(SHARED_KEY)
ciphertext = cipher.encrypt(bytes(message.encode()))
plaintext = cipher.decrypt(ciphertext)
```

2.4 How we simulated the proposed exfiltration scenario

To showcase our implementation of an one-way encrypted covert channel using ICMP, we used Docker containers, defining the following resources:

`local_net`: the network in which all the services recide. Having a shared network comes with a nice benefit: DockerDNS will be able to solve the IP addresses of the containers based on their names. So instead of providing the actual IP address, we can just work with the more user-friendly service names.

`proxy`: the service that serves as our "firewall. It is built from `alpine` linux distribution image. We give it `NET_ADMIN` priorities to be able to change network policies with `iptables`. When the container is built, it will drop all previous policies, and allow only traffic vie loopback interface, and traffic for ICMP packets.

`icmp-sender`: the service that will send the ICMP packets. It is set up to run behind `proxy`. It is also set up to use raw sockets with `cap_add: - NET_RAW`.

`icmp-listener`: is the service that listens for messages from `icmp-sender`. It is set up to operate directly on `local_net` (so not behin `proxy`), and it is also set up to use raw sockets.

`general-sender` and `general-listener`: two services that are meant to mirror the behaviour of `icmp-sender` and `icmp-listener`, but with UDP packages. Due to how `proxy` network policies are set, these two services are meant to fail to communicate.

The `readme` inside the task directory contains the instructions on how to build and run the whole suite. In short, what will happen is that `icmp-listener` will wait and recieve messages from `icmp-sender`, decrypt them and display.

On the other hand, on running `general-sender`, it will fail with a timeout due to not being able to send the non-ICMP message to `general-listener`.

3 Task 4: Mini TLS-based VPN tunneling

3.1 Prerequisites

This task covers the following topics: Virtual Private Network (VPN), TUN/TAP virtual interface, IP tunneling, routing, and TLS.

Virtual Private Network (VPN) is a private network built on top of a public network, usually the Internet. For example, it allows companies to expand their private network outside their physical location to places around the world.

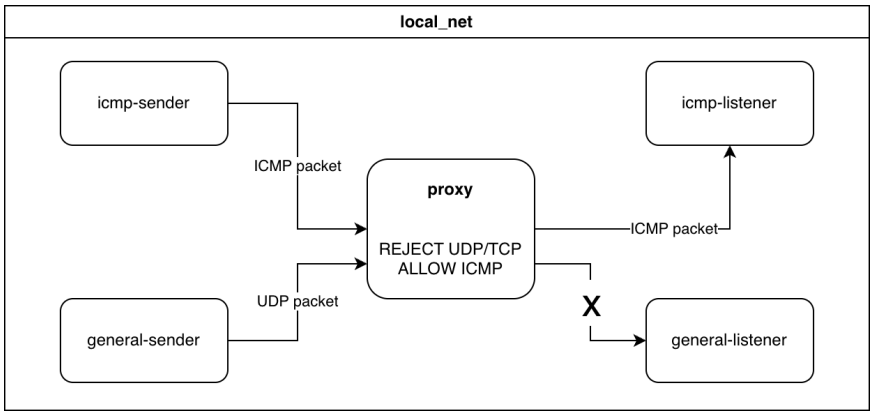

Figure 2: A scheme of how the Docker containers interact with each other.

TUN/TAP virtual interfaces are used to redirect packets to a user-space program, which can then inject them back with different values into the operating system network stack. TUN carries IP packets at the Network layer, and TAP carries Ethernet frames at the Data Link layer.

IP tunneling allows two different networks to interact. In this task, it is used to exchange messages between two hosts: one inside the network and the other using the VPN.

Routing is the process of selecting the best path for data packets to travel. Namely, it is used in this task to route traffic to the TUN interface, which is then handled by a script running locally on the machine.

TLS is a cryptographic protocol that secures communications over a computer network.

Moreover, this task is built on top of the SEED lab [1] in which VPN tunneling is created in plain text. The goal of Task 4 is to add TLS on top of it.

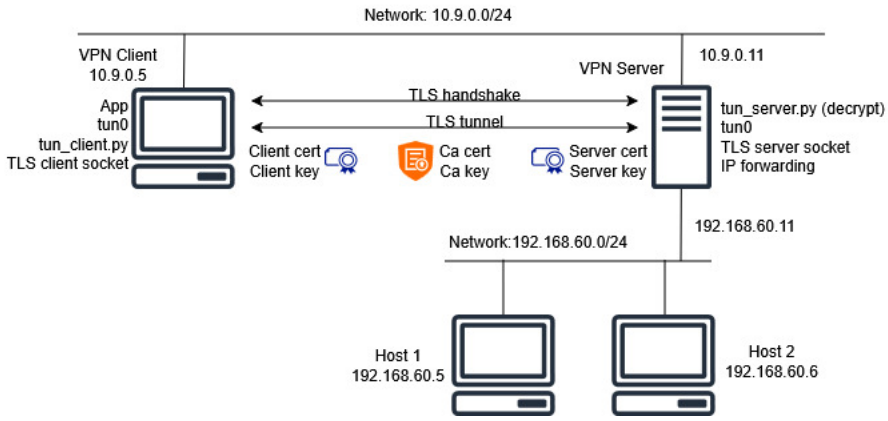

Figure 3: Network setup and operational workflow.

3.2 How we implemented the VPN Tunneling

For the connection to work, we need two different programs: `tun_client.py` running on the client side outside the network and `tun_server.py` running on the VPN server.

For both, we create a TUN interface. For simplicity, we exclude Ethernet headers and packet information for the IP packets. Next, the local certificates for each (created locally) and the Central Authority (CA) certificate are loaded.

The client creates a TCP socket, wraps it with TLS, and connects to the server. The server, after creating a TCP socket, listens to all traffic on the reserved port. When a client attempts to reach the server, they complete a TCP handshake and a TLS handshake to verify the certificates; subsequently, an encrypted socket is created (a dedicated private channel for the specific client).

The next step for the server is to redirect the encrypted traffic received from the VPN client to the network over TUN, as well as encrypt and send the local network host's responses back to the VPN client over the socket.

The tutorial for running the experiment can be found as a readme in the source code. Next, it is going to be covered the process of creating the certificates.

3.3 Certificate structure

In order to allow mutual authentication for the server and client, we must generate the respective Secure Sockets Layer (SSL) certificates. First we create a self-signed certificate according to the following discussion using the command

```
openssl req -x509 -newkey rsa:4096 -keyout ca-key.pem -out ca-
    cert.pem -days 365 -nodes -subj  /CN=vpn-ca
```

Secondly, we create a private key and a Certificate Signing Request (CSR) for the client.

```
openssl req -newkey rsa:4096 -keyout client-key.pem -out client-
    csr.pem -nodes -subj  /CN=client
```

Using the previously create root certificate, we sign CSR.

```
openssl x509 -req -in client-csr.pem -CA ca-cert.pem -CAkey ca-
    key.pem -CAcreateserial -out client-cert.pem -days 365
```

The above steps are repeated in order to create a certificate for the server as well.

3.4 Proof of Work

The conversation is between `client-10.9.0.5` and `host-192.168.60.5`.

References

[1] Wenliang Du. Vpn lab: Virtual private network lab. https://seedsecuritylabs.org/Labs_20.04/Networking/VPN/, 2026. Accessed: 2026-03-24. 5

```
host-192.168.60.6  |  * Starting internet superserver inetd                [ OK ]
host-192.18.60.5   |  * Starting internet superserver inetd                [ OK ]
server-router      | Interface Name: tun0
client-10.9.0.5    | Interface Name: tun0
client-10.9.0.5    | Error: Device for nexthop is not up.
client-10.9.0.5    | TLS connection successul
client-10.9.0.5    | From tun ==>: 0.0.0.0 --> 152.41.115.191
server-router      | Listening on TLS 0.0.0.0:9090
server-router      | From TLS <==: 0.0.0.0 --> 152.41.115.191
server-router      | From tun ==>: 0.0.0.0 --> 139.172.9.92
client-10.9.0.5    | From TLS <==: 0.0.0.0 --> 139.172.9.92
server-router      | From tun ==>: 0.0.0.0 --> 139.172.9.92
client-10.9.0.5    | From TLS <==: 0.0.0.0 --> 139.172.9.92
client-10.9.0.5    | From tun ==>: 0.0.0.0 --> 152.41.115.191
server-router      | From TLS <==: 0.0.0.0 --> 152.41.115.191
server-router      | From tun ==>: 0.0.0.0 --> 139.172.9.92
client-10.9.0.5    | From TLS <==: 0.0.0.0 --> 139.172.9.92
client-10.9.0.5    | From tun ==>: 0.0.0.0 --> 152.41.115.191
server-router      | From TLS <==: 0.0.0.0 --> 152.41.115.191
```

Figure 4: VPN Connection established between client and server.

```
root@8d14770c5768:/volumes# ./chat-connect.sh 192.168.60.5 9000
[chat-connect] 8d14770c5768 connecting to 192.168.60.5:9000
[chat-connect] Ctrl+C to exit
Connection to 192.168.60.5 9000 port [tcp/*] succeeded!
[08:49:11] f2641ff8b2eb: hey
helau
[08:50:45] f2641ff8b2eb: How are you
Good, hope anyone that reads this is happy. Good dayy
```

```
root@f2641ff8b2eb:/volumes# ./chat-listen.sh 9000
[chat-listen] f2641ff8b2eb listening on port 9000
[chat-listen] Ctrl+C to exit
Listening on 0.0.0.0 9000
Connection received on 10.9.0.5 59176
hey
[08:49:14] 8d14770c5768: helau
How are you
[08:51:00] 8d14770c5768: Good, hope anyone that reads this is happy. G
ood dayy
```

Figure 5: Encrypted message exchange between hosts.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.9.0.5	10.9.0.11	TCP	74	44130 → 9090 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2008875199 TSecr=0
2	0.000076	10.9.0.11	10.9.0.5	TCP	74	9090 → 44130 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=9514765
3	0.000087	10.9.0.5	10.9.0.11	TCP	66	44130 → 9090 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2008875199 TSecr=951476553
4	0.000308	10.9.0.5	10.9.0.11	TLSv1.3	583	Client Hello
5	0.000314	10.9.0.11	10.9.0.5	TCP	66	9090 → 44130 [ACK] Seq=1 Ack=518 Win=64768 Len=0 TSval=951476554 TSecr=2008875200
6	0.005156	10.9.0.11	10.9.0.5	TLSv1.3	3438	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Dat
7	0.005160	10.9.0.5	10.9.0.11	TCP	66	44130 → 9090 [ACK] Seq=518 Ack=3373 Win=70912 Len=0 TSval=2008875204 TSecr=951476558
8	0.009947	10.9.0.5	10.9.0.11	TLSv1.3	3216	Change Cipher Spec, Application Data, Application Data, Application Data
9	0.009952	10.9.0.11	10.9.0.5	TCP	66	9090 → 44130 [ACK] Seq=3373 Ack=3668 Win=70400 Len=0 TSval=951476563 TSecr=200887520
10	0.010164	10.9.0.11	10.9.0.5	TLSv1.3	1521	Application Data
11	0.010198	10.9.0.11	10.9.0.5	TCP	1514	9090 → 44130 [ACK] Seq=4828 Ack=3668 Win=70400 Len=1448 TSval=951476564 TSecr=200887
12	0.010279	10.9.0.5	10.9.0.11	TCP	66	44130 → 9090 [ACK] Seq=3668 Ack=6276 Win=76672 Len=0 TSval=2008875210 TSecr=951476563
13	0.010290	10.9.0.11	10.9.0.5	TLSv1.3	73	Application Data
14	0.051460	10.9.0.5	10.9.0.11	TCP	66	44130 → 9090 [ACK] Seq=3668 Ack=6283 Win=76672 Len=0 TSval=2008875251 TSecr=951476564
15	0.051483	10.9.0.11	10.9.0.5	TLSv1.3	136	Application Data
16	0.051492	10.9.0.5	10.9.0.11	TCP	66	44130 → 9090 [ACK] Seq=3668 Ack=6353 Win=76672 Len=0 TSval=2008875251 TSecr=951476605
17	0.052113	10.9.0.5	10.9.0.11	TLSv1.3	136	Application Data
18	0.097235	10.9.0.11	10.9.0.5	TCP	66	9090 → 44130 [ACK] Seq=6353 Ack=3738 Win=70400 Len=0 TSval=951476651 TSecr=200887525
19	1.400099	10.9.0.11	10.9.0.5	TLSv1.3	136	Application Data
20	1.440618	10.9.0.5	10.9.0.11	TCP	66	44130 → 9090 [ACK] Seq=3738 Ack=6423 Win=76672 Len=0 TSval=2008876640 TSecr=951477953
21	4.345386	10.9.0.5	10.9.0.11	TLSv1.3	136	Application Data
22	4.345459	10.9.0.11	10.9.0.5	TCP	66	9090 → 44130 [ACK] Seq=6423 Ack=3808 Win=70400 Len=0 TSval=951480899 TSecr=200887954
23	8.980622	10.9.0.11	10.9.0.5	TLSv1.3	136	Application Data
24	8.980656	10.9.0.5	10.9.0.11	TCP	66	44130 → 9090 [ACK] Seq=3808 Ack=6493 Win=76672 Len=0 TSval=2008884180 TSecr=951485534
25	12.540194	10.9.0.5	10.9.0.11	TLSv1.3	136	Application Data
26	12.540349	10.9.0.11	10.9.0.5	TCP	66	9090 → 44130 [ACK] Seq=6493 Ack=3878 Win=70400 Len=0 TSval=951489094 TSecr=200888773

Figure 6: Packet capture showing the TLS-encrypted tunnel traffic.